DATABASE SYSTEMS • INTERMEDIATE SQL

Modification of the Database

Joins · Views · Transactions · Integrity Constraints

INSERT / DELETE / UPDATE

JOIN OPERATIONS

VIEWS

TRANSACTIONS & CONSTRAINTS

A Practical Guide for SQL Practitioners

ROADMAP

What We'll Cover

01 **Modifying the Database**

INSERT, DELETE, and UPDATE statements

02 **Joins**

Inner, natural, and outer join operations

03 **Views**

Virtual tables, updatability, materialization

04 **Transactions**

ACID properties, commit and rollback

05 **Integrity Constraints**

Keys, checks, and referential integrity

PART 1

Modifying the Database

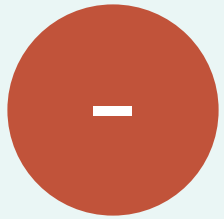
INSERT · DELETE · UPDATE

Three Ways to Change Data



INSERT

Add new tuples (rows) into a relation.



DELETE

Remove tuples that satisfy a condition.



UPDATE

Modify values of existing tuples.

All three statements operate on relations and can include a WHERE-style predicate to target specific rows.

The INSERT Statement

Adds one or more new tuples to a relation. Values must respect the relation's schema, and column order matters unless attribute names are listed explicitly.

```
INSERT INTO instructor
VALUES ('10211', 'Smith', 'Music',
45000);
```

Insert a single tuple with values listed positionally.

```
INSERT INTO course (course_id, title,
dept_name)
SELECT course_id, title, dept_name
FROM   new_courses
WHERE  credits > 3;
```

Insert using a SELECT subquery — the entire query result is inserted, row by row.

Caution: Evaluate the SELECT subquery fully before any insertion begins — otherwise a subquery referencing the same table can behave unpredictably (an insert–select recursion problem).

The DELETE Statement

Removes whole tuples from a relation — never individual attribute values. A DELETE with no WHERE clause removes every tuple in the relation.

```
DELETE FROM instructor
WHERE dept_name = 'Finance';
```

Delete all instructors belonging to the Finance department.

```
DELETE FROM instructor
WHERE salary < (
  SELECT avg(salary)
  FROM   instructor);
```

The WHERE predicate may itself contain a subquery over the same or another relation.

Note: The predicate is evaluated against the database state before any deletion — all qualifying tuples are identified first, then removed together.

The UPDATE Statement

Changes the values of one or more attributes in tuples that satisfy a condition, without altering which tuples exist.

```
UPDATE instructor
SET    salary = salary * 1.05
WHERE  salary > 100000;
```

Give every instructor earning above 100,000 a 5% raise.

```
UPDATE instructor
SET    salary = salary * 1.03
WHERE  ID IN (
  SELECT ID FROM teaches
  WHERE  semester = 'Fall');
```

Update rows selected via a subquery result set.

Order matters: SET assignments read the row's original values — all expressions in one UPDATE use the pre-update state, not values changed earlier in the same statement.

Instructor Table

ID	Name	Salary
101	Alice	5000
102	Bob	6000
103	Charlie	7000
104	David	5500

Teaches Table

ID	Course	Semester
101	DBMS	Fall
102	AI	Spring
103	OS	Fall
104	Networks	Spring

Increase the salary by **3%** for instructors whose IDs are **101** or **103**

Conditional Updates with CASE

Different tuples often need different update logic. A CASE expression lets one UPDATE statement branch per row.

```
UPDATE instructor
SET    salary = CASE
                  WHEN salary <= 100000 THEN salary * 1.05
                  ELSE salary * 1.03
                END;
```

- Lower earners (≤ 100,000) receive a larger 5% raise
- Everyone else receives a smaller 3% raise
- A single pass over the table — no need for two separate UPDATE statements

PART 2

Joins

Combining data across multiple relations

Why Do We Need Joins?

Related data is often split across multiple relations to avoid redundancy. A join recombines rows from two or more tables based on a related column.

instructor

ID	name	dept_name
10211	Smith	Music
76543	Singh	Finance

⋈ *JOIN on dept_name*

department

dept_name	building
Music	Packard
Finance	Painter

result

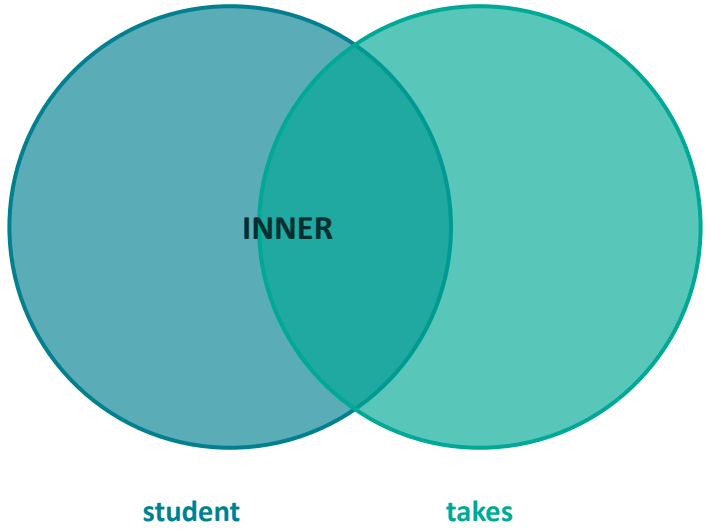
name	building
Smith	Packard
Singh	Painter

Under the hood: SQL join operators are built on the Cartesian product of the input relations, filtered by a matching condition.

INNER JOIN

Returns only the rows that have matching values in both tables. Non-matching rows from either side are excluded from the result.

```
SELECT name, course_id
FROM   student
JOIN   takes
  ON   student.ID = takes.ID;
```



Equivalent implicit form:

```
SELECT name, course_id
FROM   student, takes
WHERE  student.ID = takes.ID;
```

Student Table

ID	Name
101	Alice
102	Bob
103	Charlie
104	David

Takes Table

ID	Course_ID
101	CS101
101	CS102
102	CS103
104	CS104
105	CS105

Student: Alice, Student Bob, Student Charlie, Student David each comparing with Takes table

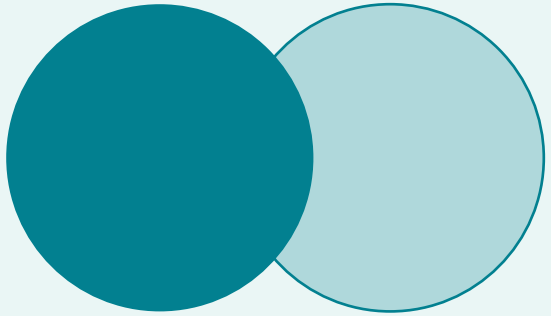
Final Result

Name	Course_ID
Alice	CS101
Alice	CS102
Bob	CS103
David	CS104

LEFT and RIGHT OUTER JOIN

Outer joins preserve rows that would otherwise be dropped by an inner join, filling unmatched columns with NULL.

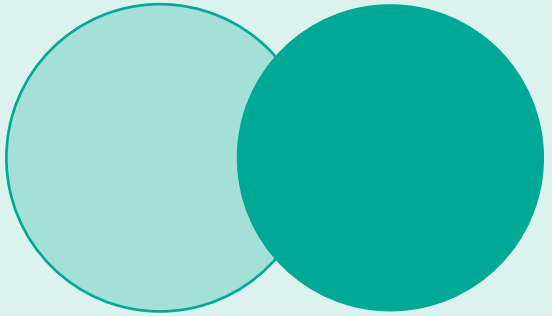
LEFT OUTER JOIN



keeps ALL rows from the left table

```
SELECT * FROM student
LEFT JOIN takes
  ON student.ID = takes.ID;
```

RIGHT OUTER JOIN



keeps ALL rows from the right table

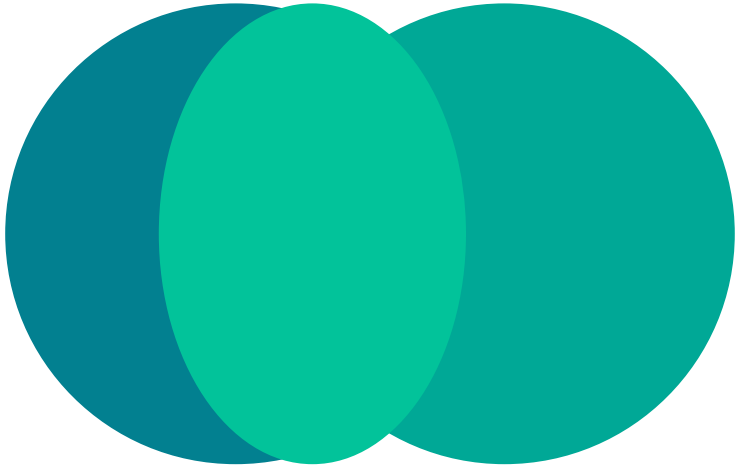
```
SELECT * FROM student
RIGHT JOIN takes
  ON student.ID = takes.ID;
```

FULL OUTER JOIN

Combines the effect of LEFT and RIGHT outer joins: every row from both tables appears at least once, with NULLs filling any unmatched side.

```
SELECT name, course_id
FROM   student
FULL OUTER JOIN takes
  ON   student.ID = takes.ID;
```

Use case: auditing data — e.g. finding students with no enrollments AND enrollments with no matching student record, in one query.



ALL ROWS **BOTH SIDES**

student table

ID	Name
1	Alice
2	Bob
3	Charlie
4	David

takes table

ID	Course
1	Database
1	AI
2	Networks
5	Physics

LEFT JOIN

Student ID	Name	Takes ID	Course
1	Alice	1	Database
1	Alice	1	AI
2	Bob	2	Networks
3	Charlie	NULL	NULL
4	David	NULL	NULL

Right JOIN

Student ID	Name	Takes ID	Course
1	Alice	1	Database
1	Alice	1	AI
2	Bob	2	Networks
NULL	NULL	5	Physics

Full JOIN

Name	Course
Alice	Database
Alice	AI
Bob	Networks
Charlie	NULL
David	NULL
NULL	Physics

Join Types at a Glance

Join Type	Unmatched Left	Unmatched Right	Typical Use
INNER JOIN	Dropped	Dropped	Only rows present in both tables
LEFT OUTER	Kept (NULLs)	Dropped	Keep every row from the primary table
RIGHT OUTER	Dropped	Kept (NULLs)	Keep every row from the secondary table
FULL OUTER	Kept (NULLs)	Kept (NULLs)	Reconcile / audit both tables

Rule of thumb: pick INNER when both sides must match; pick an OUTER join whenever "no match" is itself meaningful information.

PART 3

Views

Virtual tables built from stored queries

What Is a View?

A view is a virtual relation defined by a query. It is not stored directly — its content is computed from the underlying tables whenever it is used.



Simplicity

Hide complex joins and subqueries behind a simple name.



Security

Expose only selected columns or rows to certain users.



Reusability

Write a query once, reference it in many places.

Creating and Using a View

```
CREATE VIEW faculty AS
  SELECT ID, name, dept_name
  FROM   instructor;
```

Once defined, the view can be queried exactly like an ordinary table:

```
SELECT name
FROM   faculty
WHERE  dept_name = 'Music';
```

Behind the scenes: the database substitutes the view's stored query wherever the view name appears, then optimizes the combined query as a whole.

instructor table

ID	Name	Dept_Name	Salary
101	John	CSE	80000
102	Sarah	EEE	75000
103	David	CSE	90000
104	Emma	BBA	70000

faculty (View) table (The **Salary column is hidden)**

ID	Name	Dept_Name
101	John	CSE
102	Sarah	EEE
103	David	CSE
104	Emma	BBA

What Happens if the Original Table Changes? You **did not update the view**.

The view automatically shows the latest data because it always reads from the `instructor` table.

Why Do We Use Views?

1. **Hide sensitive information**
   - For example, don't show the `salary` column.
2. **Simplify queries**
   - Save a complex query and reuse it.
3. **Improve security**
   - Give users access only to the data they need.
4. **Provide a customized view**
   - Different users can see different subsets of the same table

PART 4

Transactions

ACID guarantees for reliable execution

What Is a Transaction?

A transaction is a unit of work made up of one or more SQL statements that must execute as a single, indivisible whole.

```
BEGIN TRANSACTION;

  UPDATE account SET balance = balance - 500 WHERE ID = 'A101';
  UPDATE account SET balance = balance + 500 WHERE ID = 'A215';

COMMIT;
```

Why it matters: if the second UPDATE fails, the transaction ensures the first is undone too — the transfer either fully happens or not at all.

COMMIT and ROLLBACK

COMMIT

Makes all changes made by the transaction permanent and visible to other transactions.

```
UPDATE account
SET balance = balance - 200
WHERE ID = 'A101';
COMMIT;
```

ROLLBACK

Undoes every change made since the transaction began, returning the database to its prior state. Cancel the changes and return to the previous state.

```
UPDATE account
SET balance = balance - 200
WHERE ID = 'A101';
ROLLBACK;
```

PART 5

Integrity Constraints

Keeping the database consistent

Core Integrity Constraints

Constraints guard the database against updates that would leave it in an invalid or inconsistent state.

`NOT NULL`

The attribute may never be left empty.

`UNIQUE`

No two tuples may share the same value.

`PRIMARY KEY`

Uniquely identifies each tuple; implies NOT NULL + UNIQUE.

`CHECK ( )`

Restricts values to a defined condition or range.

`DEFAULT`

Supplies a value automatically when none is given.

`FOREIGN KEY`

Value must exist in a referenced relation's key.

Example

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Age INT CHECK (Age >= 18),
    Department VARCHAR(50) DEFAULT 'Computer Science',
    Advisor_ID INT,
    FOREIGN KEY (Advisor_ID) REFERENCES
Instructor(Instructor_ID)
);
```

PRIMARY KEY

Student_ID	Name
1	Alice
2	Bob

UNIQUE

Student	Email
Alice	alice@gmail.com
Bob	bob@gmail.com

Foreign Keys and Referential Integrity

A foreign key ties an attribute in one relation to the primary key of another, ensuring every reference points to a row that actually exists.

Instructor_ID	Instructor_Name
101	Dr. Smith
102	Dr. John
103	Dr. David

Student_ID	Student_Name	Advisor_ID
1	Alice	101
2	Bob	102
3	Emma	101

CHECK Constraints and Assertions

A CHECK constraint restricts an attribute to values that satisfy a given predicate; an ASSERTION enforces a condition across the whole database.

```
CREATE TABLE instructor (
  ID        varchar(5),
  salary    numeric(8,2),
  CHECK (salary > 0));
```

Column-level CHECK: salary can never be zero or negative.

```
CREATE ASSERTION credits_ok
CHECK (NOT EXISTS (
  SELECT * FROM student
  WHERE tot_cred > 500));
```

Database-wide ASSERTION: no student may ever exceed 500 total credits.



DATABASE SYSTEMS

SQL: Advanced Concepts

Data Types & Schemas • Authorization • Advanced Features

Embedded & Dynamic SQL • Functions, Procedures & Triggers

A technical overview of core and advanced SQL capabilities

TOPIC 01

SQL Data Types & Schemas

Core Data Types

- **Numeric**

  INT, SMALLINT, NUMERIC(p,s), FLOAT, DOUBLE PRECISION

- **Character**

  CHAR(n), VARCHAR(n), CLOB for large text

- **Date & Time**

  DATE, TIME, TIMESTAMP, INTERVAL

- **Boolean**

  TRUE, FALSE, UNKNOWN (three-valued logic)

- **Other**

  BLOB (binary), ARRAY, ROW types in SQL:1999+

Schemas

A schema is a named collection of database objects — tables, views, domains, constraints — owned by an authorization identifier.

```
CREATE SCHEMA company
  AUTHORIZATION admin_user;

CREATE TABLE company.employee (...);
```

Catalog

A named collection of schemas forming a database instance.

TOPIC 02

Authorization

SQL's authorization model controls which users may perform which operations on which objects, using a privilege-based grant system.

1

Privileges

SELECT, INSERT, UPDATE, DELETE, REFERENCES, USAGE — granted on tables, views, or columns.

2

GRANT

```
GRANT SELECT, UPDATE ON
employee TO analyst;
Optionally WITH GRANT
OPTION to re-delegate.
```

3

REVOKE

```
REVOKE UPDATE ON employee
FROM analyst;
Can cascade to privileges
granted onward.
```

4

Roles

Named groups of privileges assigned to users — simplifies managing access at scale.

All four elements together form SQL's discretionary access control (DAC) model.

Authorization

User/Role	Privileges	SQL Command	Result
HR Staff	SELECT, UPDATE	`GRANT SELECT, UPDATE ON Employee TO HR_Staff;`	HR Staff can view and update employee records.
Intern	SELECT	`GRANT SELECT ON Employee TO Intern;`	Intern can only view employee records.
Intern	SELECT removed	`REVOKE SELECT ON Employee FROM Intern;`	Intern can no longer access the Employee table.
HR Manager (Role)	SELECT, INSERT, UPDATE	`CREATE ROLE HR_Manager;GRANT SELECT, INSERT, UPDATE ON Employee TO HR_Manager;`	Any user assigned the **HR_Manager** role automatically receives all three permissions.

Emp_ID	Name	Department	Salary
101	Alice	HR	5000
102	Bob	IT	6000
103	Charlie	Finance	7000

TOPIC 03

Advanced SQL Features

Window Functions

RANK(), ROW_NUMBER(), SUM() OVER — compute across row sets without collapsing them.

CTEs

WITH clause defines named, reusable subqueries for readable, modular queries.

Recursive Queries

WITH RECURSIVE traverses hierarchical data, e.g. org charts or bill-of-materials.

OLAP Extensions

CUBE, ROLLUP, and GROUPING SETS produce multi-level aggregate summaries.

JSON / XML Support

Native functions to store, query, and generate semi-structured data.

MERGE Statement

Combines INSERT, UPDATE, and DELETE into a single upsert operation.

1. Window Function

```
SELECT Name, Salary,
RANK() OVER(ORDER BY Salary
DESC) AS Rank
FROM Employee;
```

2. CTE

```
WITH HighSalary AS
(
  SELECT *
  FROM Employee
  WHERE Salary > 6000
)
SELECT * FROM HighSalary;
```

3. Recursive Query

```
WITH RECURSIVE Org AS (...)
SELECT * FROM Org;
```

4. OLAP (ROLLUP)

```
SELECT Department, SUM(Salary)
FROM Employee
GROUP BY ROLLUP(Department);
```

5. JSON Support

```
SELECT JSON_OBJECT('Name', Name, 'Salary', Salary)
FROM Employee;
```

6. MERGE

```
MERGE Employee
USING Employee_New
ON Employee.Emp_ID =
Employee_New.Emp_ID
...
```

Emp_ID	Name	Department	Salary	Manager_ID
101	Alice	HR	7000	NULL
102	Bob	HR	5000	101
103	Charlie	IT	6000	101
104	David	IT	4500	103
105	Emma	Finance	8000	101

Employee	Rank	High Salary (CTE)	Manager (Recursive)	Department Total (ROLLUP)	JSON Output	MERGE Result
Alice	2	Yes	Top Manager	HR = 12,000	`{"Name":"Alice","Salary":7000}`	No Change
Bob	4	No	Reports to Alice	HR = 12,000	`{"Name":"Bob","Salary":5000}`	**Salary Updated → 5500**
Charlie	3	No	Reports to Alice	IT = 10,500	`{"Name":"Charlie","Salary":6000}`	No Change
David	5	No	Reports to Charlie	IT = 10,500	`{"Name":"David","Salary":4500}`	No Change
Emma	1	Yes	Reports to Alice	Finance = 8,000	`{"Name":"Emma","Salary":8000}`	No Change
Frank	—	—	—	—	—	**New Employee Inserted**

TOPIC 04

Embedded SQL

SQL statements are embedded directly inside a host language (C, Java, COBOL). A pre-compiler translates them into host-language calls before normal compilation.

- Statements prefixed with EXEC SQL
- Host variables share data between SQL and code, marked with a colon (:var)
- `EXEC SQL BEGIN DECLARE SECTION`- The variables declared below will be used by both the C program and the SQL statements.
- EXEC SQL END DECLARE SECTION is the end of the variable declarations used by Embedded SQL
  `EXEC SQL` says the following statement is SQL, not C code.

Trade-off: fast and type-checked at compile time, but the SQL text itself cannot change at run time.

A C program wants to display the name of a student whose ID is **101**

C host program

```
#include <stdio.h>
EXEC SQL BEGIN DECLARE SECTION;
int student_id = 101;
char student_name[50];
EXEC SQLEND DECLARE SECTION;

EXEC SQL
SELECT Name
INTO :student_name
FROM Student
WHERE Student_ID =:student_id;

printf("Student Name: %s", student_name);
```

TOPIC 06

Functions & Procedures

Stored Function

Returns a single value; can be called inside a SQL expression, e.g. SELECT bonus(emp_id).

```
CREATE FUNCTION bonus(emp_id INT)
RETURNS DECIMAL(9,2)
BEGIN
  DECLARE b DECIMAL(9,2);
  SELECT salary * 0.1 INTO b
    FROM employee
    WHERE id = emp_id;
  RETURN b;
END;
```

Stored Procedure

Performs actions and may return zero, one, or multiple values via OUT parameters; called with CALL.

```
CREATE PROCEDURE raise_salary
  (IN emp_id INT, IN pct DECIMAL)
BEGIN
  UPDATE employee
  SET salary = salary * (1 + pct)
  WHERE id = emp_id;
END;

CALL raise_salary(101, 0.05);
```

TOPIC 07

Triggers

A trigger is a stored program that fires automatically in response to a data-modification event on a table.

01

Event

INSERT, UPDATE, or DELETE on a specified table

02

Timing

BEFORE or AFTER the triggering event occurs

03

Granularity

FOR EACH ROW or FOR EACH STATEMENT

04

Action

PL/SQL-style block runs, using OLD and NEW row values

```
CREATE TRIGGER audit_salary
BEFORE UPDATE ON employee
FOR EACH ROW
WHEN (NEW.salary <> OLD.salary)
BEGIN
  INSERT INTO salary_log VALUES (OLD.id, OLD.salary, NEW.salary, SYSDATE);
END;
```

Employee Table (Before Update)

ID	Name	Salary
101	Alice	5000
102	Bob	6000
103	Charlie	7000

Salary Log Table (Initially Empty)

ID	Old Salary	New Salary	Date
(Empty)			

The Trigger Runs Automatically

OLD.salary = 5000
NEW.salary = 5500

Someone Updates Alice's Salary

ID	Salary
101	5000

ID	Salary
101	5500

Salary Log Table (After Trigger)

ID	Old Salary	New Salary	Date
101	5000	5500	05-Aug-2026